

PORTADA

Técnicas de Programación

Unidad 2 — Paradigma Objetual: Clases, Objetos y Herencia

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



APERTURA

**Hasta ahora tu código decía "haz esto".
Desde hoy, tu código dice "esto *es* algo".**



CONCEPTO

Del paradigma imperativo al objetual

En la Unidad 0 organizaste código como una **secuencia de instrucciones sobre variables sueltas**. El **paradigma orientado a objetos (POO)** propone otra unidad de organización: agrupar los datos y el comportamiento que les pertenece en una sola entidad, llamada **objeto**.

Un objeto modela una "cosa" del problema real: un estudiante, una cuenta bancaria, un carro. **Tiene características (qué sabe) y comportamientos (qué hace)**.

CONCEPTO

Clase: la plantilla del objeto

Una **clase** es la plantilla (el "molde") que describe qué atributos y qué métodos va a tener cada objeto que se construya a partir de ella. Un **objeto** es una **instancia concreta** de esa clase, con valores propios en sus atributos.

Clase	Objeto (instancia)
El plano de una casa	Una casa construida a partir de ese plano
Estudiante	ana , con nombre "Ana" y código "2508307-1"

Una sola clase puede producir cualquier cantidad de objetos distintos, cada uno con sus propios valores.

CONCEPTO

Atributos y métodos

Los **atributos** (también llamados campos o propiedades) son las variables que guardan el estado de un objeto. Los **métodos** son las funciones que definen su comportamiento — casi siempre operan sobre esos mismos atributos.

```
public class Estudiante {  
    private String nombre;    // atributo  
    private String codigo;    // atributo  
  
    public void saludar() {    // método  
        System.out.println("Hola, soy " + nombre);  
    }  
}
```

`private` significa que el atributo **solo es accesible dentro de la propia clase** — es la base del encapsulamiento, que veremos con más detalle en la próxima unidad.

CONCEPTO

El constructor

Un **constructor** es un método especial que se ejecuta automáticamente al crear un objeto con `new` : se usa para dejar el objeto en un estado inicial válido, típicamente asignando valores a sus atributos.

```
public class Estudiante {  
    private String nombre;  
    private String codigo;  
  
    public Estudiante(String nombre, String codigo) {  
        this.nombre = nombre;  
        this.codigo = codigo;  
    }  
}
```

`this` se refiere **al objeto que se está construyendo** — distingue el atributo de la clase (`this.nombre`) del parámetro recibido (`nombre`).

CONTRA EJEMPLO

El constructor "gratis" no siempre alcanza

Si una clase no declara ningún constructor, Java genera uno vacío llamado **constructor por defecto**, que no recibe parámetros y deja los atributos en sus valores iniciales (`null` , `0` , `false`).

```
Estudiante e = new Estudiante(); // válido solo si no escribiste
                                   // NINGÚN constructor propio
```

Error común: en cuanto defines un constructor con parámetros, el constructor por defecto **desaparece** — si además quieres poder crear objetos sin argumentos, debes escribir ese constructor vacío tú mismo.

CONCEPTO

Instanciar objetos con new

Crear un objeto a partir de una clase se llama **instanciar**. La palabra clave `new` reserva memoria para el objeto y llama al constructor indicado.

```
Estudiante ana = new Estudiante("Ana", "2508307-1");  
Estudiante luis = new Estudiante("Luis", "2508307-2");  
  
ana.saludar(); // Hola, soy Ana  
luis.saludar(); // Hola, soy Luis
```

ana y luis son dos objetos distintos, contruidos con la misma clase pero con sus propios valores de atributos — cada uno vive en su propio espacio de memoria.



INTERACCIÓN

¿Clase u objeto?

Clasifiquen cada elemento como "clase" o "objeto":

1. CuentaBancaria
2. `CuentaBancaria miCuenta = new CuentaBancaria(...)`
3. El plano de una fábrica de carros
4. Un carro específico que sale de esa fábrica, con placa propia

5 minutos · respondan en el chat antes de revisar juntos

TRANSICIÓN

¿Y si dos clases comparten casi todo?

Un `Estudiante` y un `Profesor` comparten nombre, código y correo — pero cada uno agrega algo propio (semestre matriculado vs. asignaturas a cargo). **Copiar y pegar esos atributos en cada clase** funciona, pero duplica código y duplica errores cuando algo cambia. La POO tiene un mecanismo para esto: la **herencia**.

CONCEPTO

Herencia: reutilizar una clase existente

La **herencia** permite que una clase (**subclase**) reciba automáticamente los atributos y métodos de otra clase ya existente (**superclase**), y además agregue o modifique lo que necesite.

Modela una relación "**es un/a**": un Estudiante "es una" Persona ; un Profesor "es una" Persona . Si la relación no se puede leer de forma natural como "es un/a", probablemente **no** es un caso de herencia.



CONCEPTO

Sintaxis: extends y super()

```
public class Persona {
    protected String nombre;
    public Persona(String nombre) { this.nombre = nombre; }
    public void saludar() { System.out.println("Hola, soy " + nombre); }
}

public class Estudiante extends Persona {
    private String codigo;
    public Estudiante(String nombre, String codigo) {
        super(nombre);      // llama al constructor de Persona
        this.codigo = codigo;
    }
}
```

`extends` declara la relación de herencia; `super(...)` invoca el constructor de la superclase, **siempre como primera línea** del constructor de la subclase.

CONCEPTO

protected vs private

Un atributo `private` en `Persona` **no sería visible** desde `Estudiante` , aunque herede de ella. `protected` es un nivel intermedio: visible dentro de la propia clase, sus subclases, y otras clases del mismo paquete.

Modificador	¿Visible desde una subclase?
<code>private</code>	No
<code>protected</code>	Sí
<code>public</code>	Sí (y desde cualquier otra clase)

CONTRAEJEMPLO

Cuando la herencia no es la respuesta

Modelar `Auto extends Motor` es un error común: un auto **no "es un" motor**, sino que **"tiene un" motor**. Esa segunda relación se llama **composición** (un atributo de tipo `Motor` dentro de `Auto`), no herencia.

```
// Incorrecto: Auto no "es un" Motor
public class Auto extends Motor { }

// Correcto: Auto "tiene un" Motor (composición)
public class Auto {
    private Motor motor;
}
```

La prueba rápida: si no puedes completar la frase "X es un/a Y" de forma natural, no uses `extends` .

CONCEPTO

Sobrescribir comportamiento: @Override

Una subclase puede **redefinir** un método heredado si necesita un comportamiento distinto, usando la misma firma (nombre y parámetros) que el método de la superclase. La anotación `@Override` no es obligatoria, pero **documenta la intención** y hace que el compilador avise si la firma no coincide exactamente.

```
public class Profesor extends Persona {  
    @Override  
    public void saludar() {  
        System.out.println("Buen día, profesor " + nombre);  
    }  
}
```

INTERACCIÓN

Diseñen la jerarquía

En parejas, definan en papel (sin codificar todavía) una jerarquía de clases para un sistema de biblioteca:

- Una superclase `MaterialBibliografico` con los atributos que compartan todos los materiales.
- Dos subclases, `Libro` y `Revista`, cada una con al menos un atributo propio.

8 minutos · estén listos para justificar por qué cada relación "es un/a"

SÍNTESIS

Lo que hay que llevarse de hoy

1. Una clase es la plantilla; un objeto es una instancia concreta creada con `new` .
2. Atributos guardan el estado; métodos definen el comportamiento; el constructor deja el objeto en un estado inicial válido.
3. La herencia (`extends`) reutiliza atributos y métodos de una superclase cuando la relación es genuinamente "es un/a" — de lo contrario, es composición.
4. `super(...)` invoca el constructor de la superclase; `@Override` documenta que un método redefine el heredado.



CIERRE

Antes de la próxima sesión

- Escribir en el IDE la jerarquía `Persona / Estudiante / Profesor` de esta sesión y probar que `saludar()` se comporta distinto en cada subclase.
- Pensar en dos métodos que, en la vida real, "hacen lo mismo" pero reciben datos distintos (ej. sumar dos enteros vs. sumar dos decimales) — es el punto de partida de la próxima clase.

La próxima sesión continúa dentro de la Unidad 2 con la sobrecarga de métodos: mismo nombre de método, distintas listas de parámetros.

